

Guide de fonctionnement technique

Public : équipe IT

1. Architecture du projet

Le projet contient plusieurs fichiers :

- index.html : Structure de la page web.
- script.js

Contient toute la logique JavaScript :

- chargement du modèle IA ;
- détection des objets ;
- création de l'inventaire ;
- lecture vocale.
- style.css : Gère le style visuel de l'application.
- semantic.json : Contient les traductions, catégories et définitions des objets détectés.
- assets : Contient les photos à tester sur le modèle.

2. Fonctionnement général

Le pipeline du projet est :

```
image
↓
détection IA
↓
prédictions
↓
inventaire
↓
JSON sémantique
↓
affichage
↓
lecture vocale
```

3. Chargement du modèle IA

Le projet utilise :

TensorFlow.js

et le modèle :

COCO-SSD

Le modèle est chargé avec :

```
model = await cocoSsd.load();
```

4. Détection des objets

Quand l'utilisateur clique sur :

Analyser l'image

la fonction suivante est exécutée :

```
const predictions = await model.detect(image);
```

Le modèle retourne une liste d'objets détectés.

Exemple :

```
[  
  { class: "car", score: 0.92 },  
  { class: "person", score: 0.87 }  
]
```

5. Création de l'inventaire

Le projet compte les objets par classe.

Exemple :

```
{  
  car: 1,  
  person: 4,  
  dog: 1  
}
```

Le comptage est réalisé avec :

```
inventaire[objet] = (inventaire[objet] || 0) + 1;
```

6. Gestion du JSON sémantique

Le fichier :

semantic.json

permet d'ajouter :

- une traduction ;
- une catégorie ;
- une définition.

Exemple :

```
{  
  "car": {  
    "fr": "voiture",  
    "categorie": "transport",  
    "definition": "Une voiture est un véhicule utilisé pour se déplacer."  
  }  
}
```

```
}  
}
```

Le fichier JSON est chargé avec :

```
semanticData = await response.json();
```

7. Affichage des résultats

Les résultats sont affichés dans la page avec :

innerHTML

Exemple affiché :

1 voiture

Catégorie : transport

Définition : Une voiture est un véhicule utilisé pour se déplacer.

8. Lecture vocale

Le projet utilise :

Web Speech API

pour lire le résultat à voix haute.

La lecture est réalisée avec :

```
speechSynthesis.speak(utterance);
```

9. Technologies utilisées

- HTML
- Structure de la page.
- CSS
- Style visuel.
- JavaScript
- Logique de l'application.
- TensorFlow.js
- Bibliothèque IA utilisée dans le navigateur.
- COCO-SSD
- Modèle pré-entraîné de détection d'objets.
- JSON
- Stockage des données sémantiques.
- Web Speech API
- Lecture vocale.

Conclusion

Cette application combine intelligence artificielle, JavaScript et accessibilité vocale afin de transformer une image en inventaire lisible et audible.